Tamper Evident Logging System

Cybersecurity and Network Security Internship Assessment

Submitted By : Morabad Riya Shivanand

Date : 02 March 2026

Table of Contents

1. Introduction 3
2. System Design 3
3. Implementation 4
4. Testing and Results 7
5. Analysis of Results 8
6. Conclusion 8
7. References 9

1. Introduction

Logs are an integral component of today's computer systems as they record various systems activities including login trials, transactions and user actions that are useful for system administrators to monitor systems' behaviour as well as investigate security breaches However, there are chances that attackers may try to modify or delete logs to hide their malicious activities. To overcome this problem tamper evident logging systems are used. These systems make sure that once a log entry is made it cannot be modified without detection. A tamper-evident logging system was developed in this project, using Python programming language, where each log entry is linked using SHA-256 cryptographic hashing; if any entry is modified, deleted, or reordered, the system will be able to detect and recognize this change as it will break the integrity of the log chain.

2. System Design

The system implementation uses an architecture based on a hash-linked log structure; this is the same concept used in blockchain systems. Each log entry is cryptographically linked with the previous one, so it forms a chain of entries ensuring that the integrity of the log history is maintained. Every log entry contains the following fields:

- Index – position of the log entry in the chain
- Timestamp – time when the event occurred
- Event Type – category of the event like LOGIN, LOGOUT or TRANSACTION
- Description – additional information about the event
- Previous Hash – hash of the previous log
- Entry Hash – SHA-256 hash calculated from the current entry data The Entry Hash, also produced by data's hashing algorithm SHA-256, comprises the information of log entries as well as the previous hash. Given that each entry is based on the preceding one any alteration in earlier record blocks effectively destroys the chain's integrity.

3. Implementation

The tamper-evident logging system was implemented using the Python programming language due to its simplicity, readability and availability of built-in libraries for cryptographic operations and file handling. The system relies on a number of Python libraries such as hashlib, json, datetime where hashlib is used to provide the SHA-256 hash algorithm for creating secure hash values for log entries, json is to store log entries in structured format and datetime module is to generate timestamp for each event. Hash Computation A dedicated function was created to compute SHA-256 hash of every log entry. When the log entry data is provided as input, the function internally serializes that data into JSON format due to few benefits offered by the JSON format. One benefit which we are going to use here is JSON always stores the keys in lexicographical order. The serialized data is passed to SHA-256 algorithm which produces a fixed length hash value. The hash produced is used as the unique fingerprint of the log entry.

Adding Log Entries

The system allows users to add new log entries through a menu-driven interface. When a user selects the option to add a log entry, the program follows these steps:

1. Load the existing log chain from the JSON file.
2. Retrieve the hash of the previous entry.
3. Create a new entry with index, timestamp, event type, and description.
4. Compute the SHA-256 hash of the entry.
5. Append the entry to the log chain.
6. Save the updated chain back to the file.

This process ensures that each new entry is securely linked to the previous one.

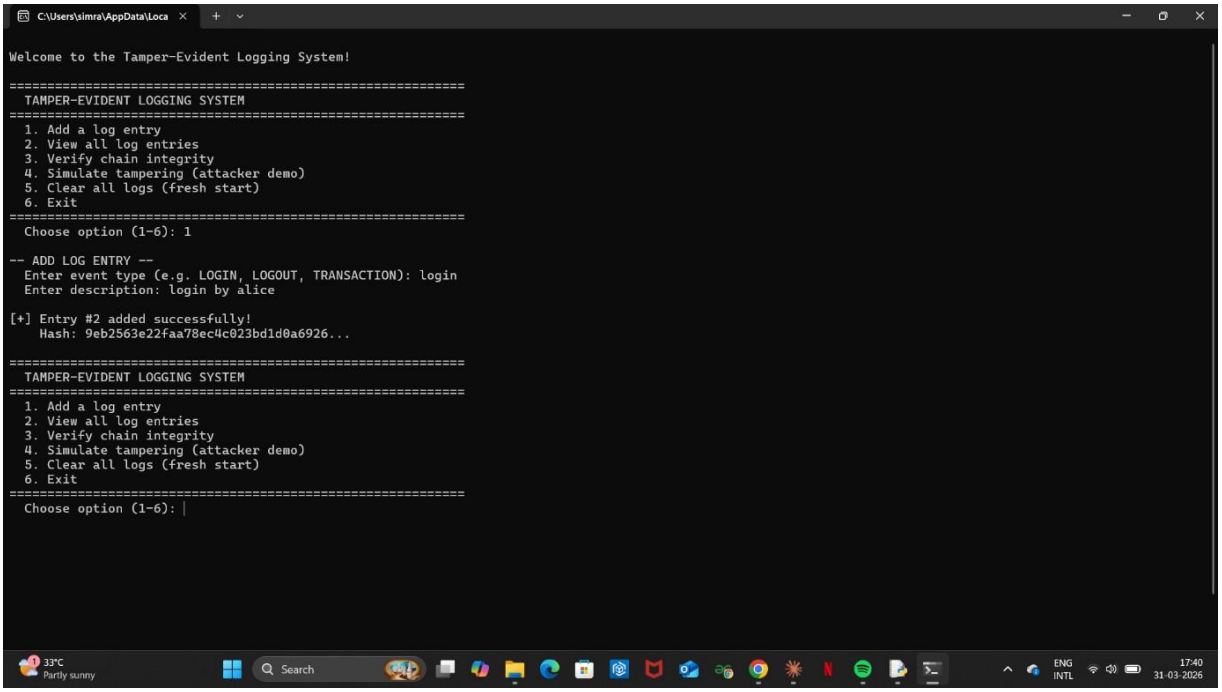

Figure 1 : Adding Log Entries

Viewing Log Entries

The system also lets users view all stored log entries. When this option is selected, the program reads the JSON log file and prints the details of each entry in an easy-to-read format.

This feature helps users check the log history and verify that entries are correctly linked.

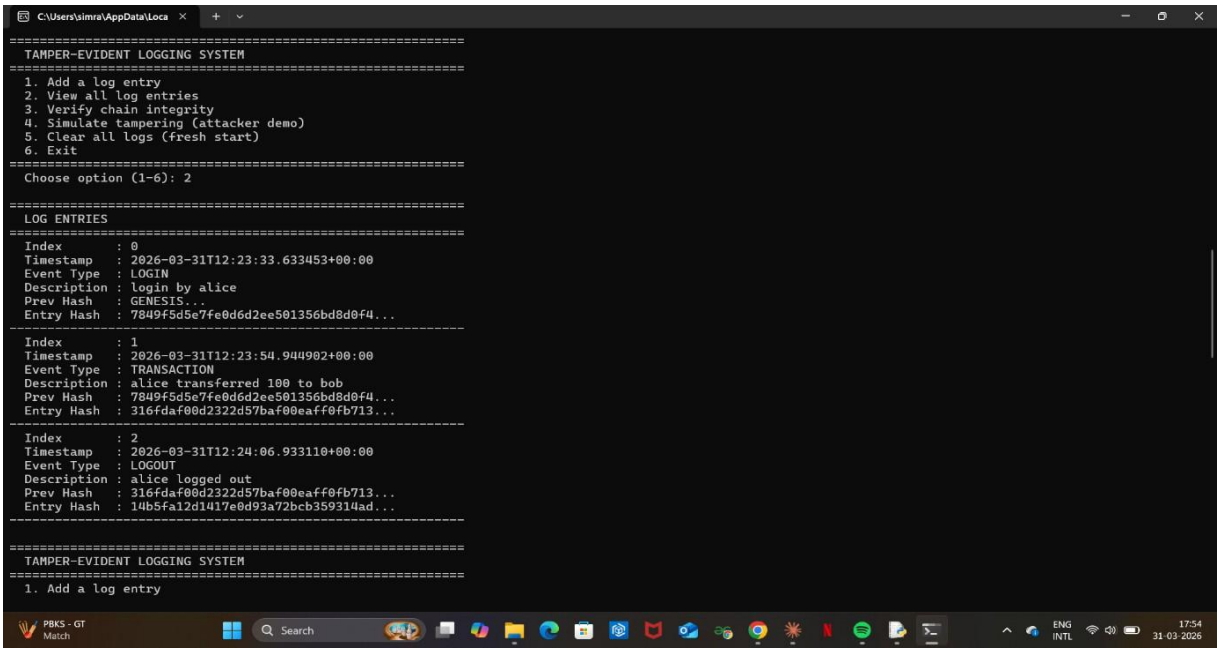

Figure 2 : Viewing Log Entries

Chain Verification

The verification function checks whether the log chain has been changed. It recalculates the hash for each entry and compares it with the stored hash value.

The function also checks if the previous hash field matches the hash of the last entry. If either condition is not met, the system reports a tampering event.

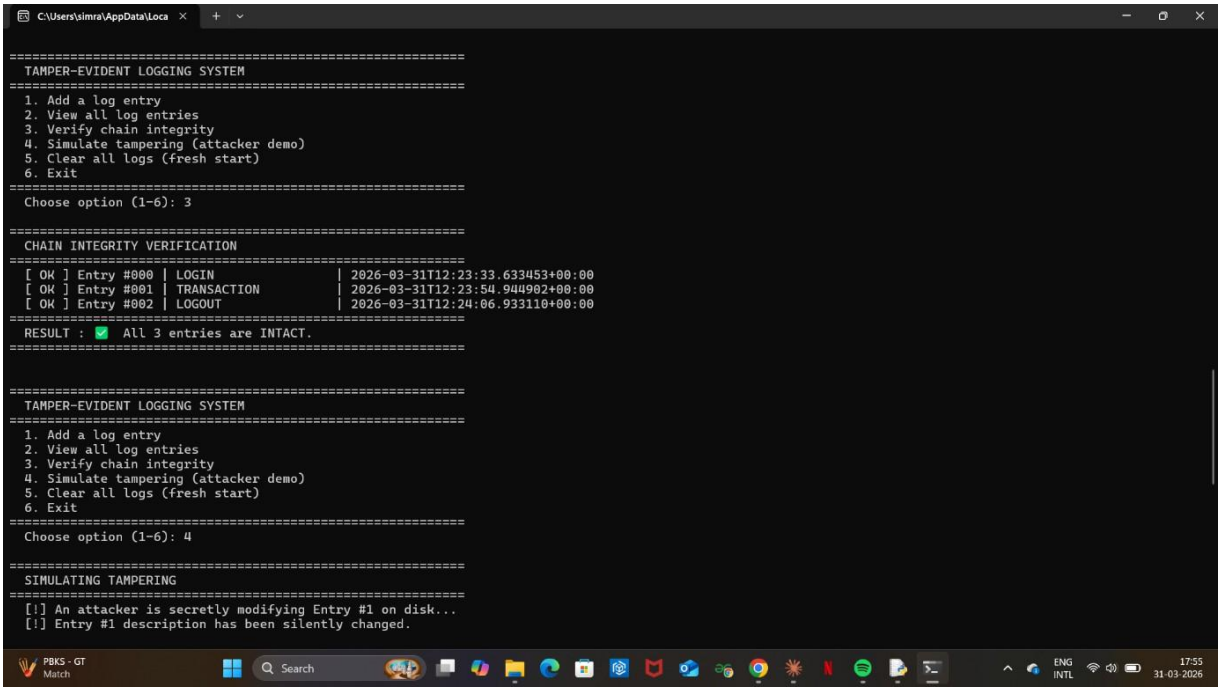

Figure 3 : Chain Verification

Tampering Simulation

To demonstrate how well the tamper-evident mechanism works, a tampering simulation feature was added. This feature modifies a log entry directly in the JSON file without updating its hash value.

When the verification process runs after this change, the system detects the mismatch and reports that the log chain's integrity has been compromised.

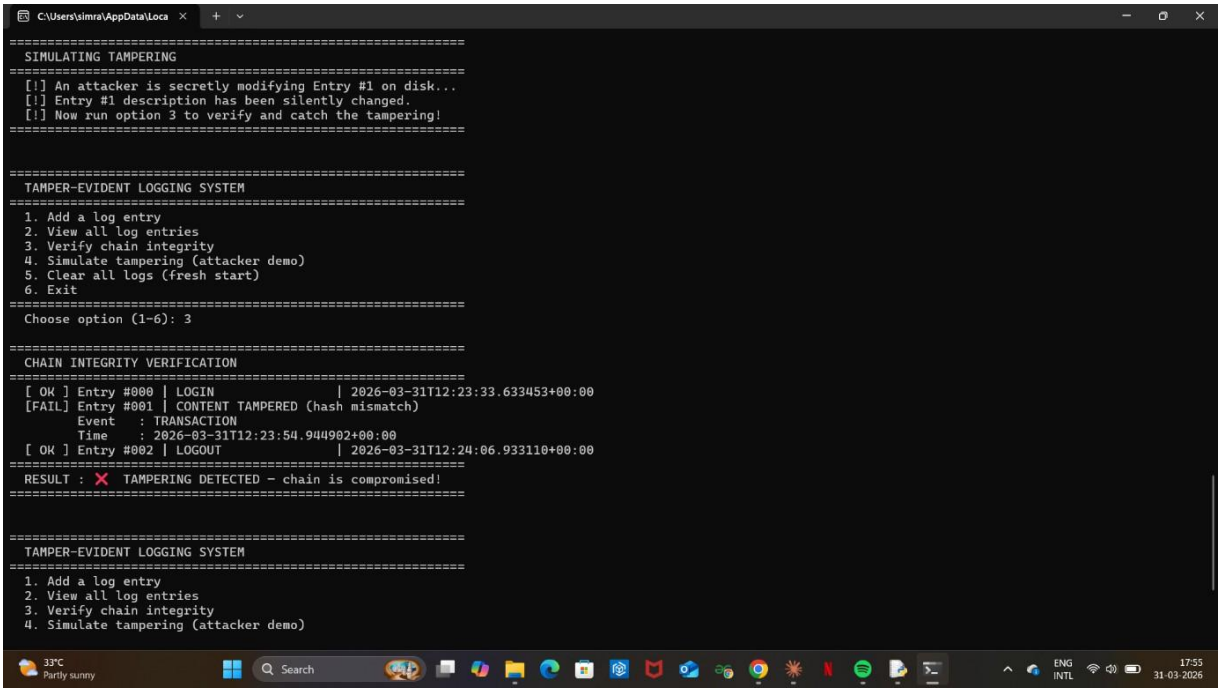

Figure 4 : Tampering Simulation

4. Testing and Results

The system was evaluated by generating multiple log entries that represented various system events. Examples included login attempts, transactions, and logout activities. These entries were successfully saved in the log file and connected using cryptographic hashes.

Once several entries were created, the log chain verification process was carried out. The system effectively verified that all entries were complete and the chain's integrity was valid.

To test the tamper-evident mechanism, the tampering simulation function was utilized. This feature changed the description field of a log entry without updating its associated hash value.

When the verification process was run once more, the system identified a discrepancy between the stored hash and the newly calculated hash for the altered entry. The program indicated that the chain had been breached and pinpointed the impacted entry.

These findings show that the system effectively identifies any unauthorized changes made to log entries.

5. Analysis of Results

The tamper-evident logging system was evaluated with various sample events, including user login, transaction activity, and logout actions. Each event was accurately logged in the file, including a timestamp, type of event, and a description. When new entries were added, the system created a SHA-256 hash for each log entry and connected it to the prior entry using the previous hash field. This formed an ongoing sequence of log entries that maintained the log history's integrity. During regular operation, the verification process ensured that every entry in the chain was legitimate. The computed hashes corresponded to the saved hashes, and the prior hash values accurately linked to the previous entries. This showed that the hashing method and chain structure were operating as intended. To assess the system's security, the tampering simulation function was utilized to alter the description of a current log entry in the JSON file. Since the entry hash wasn't updated following this change, the chain's integrity was compromised. When the chain verification process was run once more, the system identified a discrepancy between the saved hash and the newly computed hash. The program indicated that there had been interference and recognized it.

6. Conclusion

This project effectively set up a tamper-evident logging system by utilizing Python along with SHA-256 cryptographic hashing. The system logs events like login attempts, transactions, and user activities, making sure that the log entries remain intact and unaltered. By connecting each log entry to the previous one using cryptographic hashes, the system forms a secure chain that safeguards the log history against unauthorized changes.

The test results showed that the system operates properly under normal conditions and is capable of effectively identifying attempts at tampering. When a log entry was altered during the tampering simulation, the verification system detected a discrepancy in the hash chain and indicated that the log's integrity had been breached.

Even though the system is a simplified prototype, it illustrates the core principles applied in secure audit logging systems. With further enhancements like remote storage, encryption, and authentication methods, the system can be better suited for real-world security applications.

7. References

Python Documentation – https://docs.python.org
SHA-256 Hash Algorithm – https://nvlpubs.nist.gov
JSON Documentation – https://www.json.org